CÓDIGOS DE SAÍDA

Duas linhas corretas isoladamente: `cd /var/www/releases/atual` e `rm -rf *`. No dia em que `atual` virou link quebrado, o `cd` falhou — e o `rm` rodou no diretório em que o script tinha começado. Faltava um `&&`.

056
GUIAS LINUX

O MAPA DOS 256 VALORES — 0 É SUCESSO, O RESTO DIZ *QUAL* FALHA

0 sucesso	**1** erro genérico	**2** uso incorreto — sintaxe, opção inválida	**64–78** convenção do `sysexits.h`	**126** achou, mas não é executável	**127** comando não encontrado
128+N morto pelo sinal N	**130** Ctrl+C — SIGINT (2)	**137** SIGKILL (9) — o OOM killer	**143** SIGTERM (15) — parada de serviço	**255** fora de faixa, ou erro do `ssh`	**$?** o último — sobrescrito a cada comando

A faixa de 126 a 143 é a que mais economiza diagnóstico: um serviço que sai com **137** não tem defeito de lógica — foi morto pelo kernel, quase sempre por falta de memória (procure no `dmesg`, não no código). Um **127** é o `PATH` sem o diretório certo; um **126** é bit `x` ausente ou *shebang* errado. **O código é um byte:** `exit 256` vira **0** e `exit -1` vira **255** — contador de erros não serve como código de saída. E `$?` é sobrescrito pelo comando seguinte, *inclusive pelo `echo` que o imprime*: guarde em `rc=$?` se for usar duas vezes. Já `exit` sem argumento propaga o `$?` do último comando — o comportamento certo no fim de um bloco de tratamento de erro.

CURTO-CIRCUITO · E A ARMADILHA DO FALSO TERNÁRIO

`A && B || C`
o caso feliz: **`A` falha**

A falhou → **B** pulado pelo `&&` → **C** executa — o `||` viu a falha ✓

`A && B || C`
a armadilha: **`A` passa e `B` falha**

A sucesso → **B** executa **e falha** → **C** executa **também** ✗

A forma correta quando B pode falhar: `if A; then B; else C; fi`. O `||` não faz ideia de que o `&&` existiu — ele só observa o código do que veio *imediatamente* antes. Enquanto `B` for algo que não falha (`echo`, atribuição, `true`), a cadeia é idiomática e segura.

`;` e a **quebra de linha** significam a mesma coisa: *execute o próximo, aconteça o que acontecer*. Para agrupar, `cmd || { erro "falhou"; exit 1; }` roda no **shell atual** (espaço depois de `{` e `;` antes de `}` são obrigatórios); já `cmd || ( erro; exit 1 )` cria **subshell** — o `exit` encerra só o subshell e **o script segue**, com o erro que deveria abortá-lo não abortando nada.

O *PIPELINE* MENTE · O `SET -E` TEM EXCEÇÕES

kali@kali: ~/lab-exit

```
$ tar czf - /naoexiste | gzip > b.gz; echo $?
0 # o backup "deu certo" e saiu vazio
$ set -o pipefail       # devolve o 1o que falhou
$ false | true; echo $?         1
$ false | true | false
$ echo "${PIPESTATUS[@]}"       1 0 1
# PIPESTATUS tambem e sobrescrito: copie antes de usar
$ readonly EX_USO=2 EX_DEP=3 EX_PERM=4
$ command -v jq >/dev/null || exit "$EX_DEP"
```

SET -E IGNORA A FALHA EM...

```
if cmd; then … fi
while cmd; do … done
cmd || true
cmd && outro
! cmd
```

Não são caprichos: em todas elas a falha **faz parte da lógica escrita**. Duas consequências pegam quase todo mundo: **1.** uma função chamada dentro de `if` roda **inteira** sem `errexit` — só o retorno final é avaliado; **2.** o **último** comando de uma função vira o retorno dela, e um `(( n ))` com `n=0` reporta falha que nunca houve. Termine toda função com `return 0` explícito.

Uma quebra de linha entre dois comandos é um `;` — e num script de limpeza isso é o desastre inteiro. `cd "$DIR"` numa linha e `rm -rf ./*` na seguinte apaga o diretório *atual* se o `cd` falhar. A correção cabe em dois caracteres: `cd "$DIR" && rm -rf ./*` — melhor ainda com `set -euo pipefail` no topo e `echo` na frente no primeiro ensaio. Prefira `./*` a `*`: um arquivo chamado `-rf` ou `--no-preserve-root` deixa de virar opção. E `rm -f` engole o erro sem avisar — foi ele que fez o script do cenário de abertura terminar com cara de sucesso.

RESUMO RÁPIDO

- ✓ **0 é sucesso, o resto diz qual falha.** `$?` guarda só o último — copie em `rc=$?` antes de reusar.
- ✓ **As faixas têm significado:** 126 achou/não executa · 127 não achou · `128+N` morte por sinal (130 Ctrl+C, 143 SIGTERM, 137 SIGKILL/OOM).
- ✓ **`&&` e `||` são curto-circuito**, e `A && B || C` **não** é um ternário: se `B` falhar, `C` roda também.
- ✓ **`{ …; }` agrupa no shell atual; `( … )` cria subshell** — e o `exit` lá dentro não encerra o script.
- ✓ **Um *pipeline* devolve o código do último.** `set -o pipefail` ou `PIPESTATUS`; sem um dos dois, um `tar | gzip` com o `tar` quebrado reporta sucesso.
- ✓ **Dê nome aos seus códigos** (`EX_USO=2`, `EX_DEP=3`): quem chama distingue "faltou dependência" de "argumento errado".

NO KALI

Muita ferramenta usa código ≠ 0 para "não encontrei nada" — e isso não é erro. O `grep` devolve **1** quando não casa e **2** quando houve erro de verdade; o `diff` devolve 1 para "arquivos diferentes"; o `find` devolve 0 mesmo sem achar nada. Sob `set -e`, um `grep` que não casa **aborta o script** — corrija com `|| true` ou teste com `if`. **O `nmap` sai com 0 mesmo quando o alvo não responde**: para ele, "varri e não achei porta aberta" é sucesso, então decidir o próximo passo pelo código de saída decide errado — use a saída estruturada (`-oX`, `-oG`) analisada depois; o mesmo para `nikto`, `gobuster` e `hydra`, cujos códigos variam entre versões num *rolling*. `systemctl is-active --quiet ssh` foi feito para ser usado por código, não por texto — a palavra pode mudar de idioma e existe o estado `activating`. **O `ssh` devolve o código do comando remoto, mas usa 255 para os próprios erros** (autenticação, host inalcançável): num laço sobre hosts, trate 255 como problema de conexão, registre e siga. E registrar comando, horário e código por alvo é parte da **trilha de auditoria** — é o que sustenta depois a demonstração de que o trabalho ficou dentro do escopo contratado (Lei 12.737/2012 vista pelo lado de quem tem autorização e precisa comprová-la).